Colab is now part of Google AI Plans. Get upgraded compute power and a suite of new AI tools all in one bundle. Explore the Plans

Phonological Awareness Risk Identification Model

upload the data set

```
from google.colab import files
uploaded = files.upload()
```

```
Choose Files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving ml_ready_report.csv to ml_ready_report.csv
```

Read the CSV file

```
import pandas as pd

file_name = "ml_ready_report.csv"
df = pd.read_csv(file_name)

print("Shape:", df.shape)
df.head()
```

```
Shape: (49, 17)
```

	pa_score	pa_time	vp_score	vp_time	ls_score	ls_time	total_time	pa_efficiency	vp_efficiency	ls_efficiency	avg_score	avg_time	score_range	tim
0	20	11.13	50.000000	11.21	50	17.24	192	1.796945	4.460303	2.900232	40.000000	13.193333	30.000000	
1	0	24.28	16.666667	13.52	75	15.00	263	0.000000	1.232742	5.000000	30.555556	17.600000	75.000000	
2	20	10.65	33.333333	13.30	50	10.59	175	1.877934	2.506265	4.721435	34.444444	11.513333	30.000000	
3	60	23.03	33.333333	22.14	50	12.68	299	2.605297	1.505571	3.943217	47.777778	19.283333	26.666667	
4	40	22.15	50.000000	31.34	25	18.65	373	1.805869	1.595405	1.340483	38.333333	24.046667	25.000000	

Inspect the data set

```
print("Columns:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

print("\nRisk level distribution:")
print(df["risk_level_encoded"].value_counts().sort_index())
```

```
Columns:
['pa_score', 'pa_time', 'vp_score', 'vp_time', 'ls_score', 'ls_time', 'total_time', 'pa_efficiency', 'vp_efficiency', 'ls_efficiency', 'avg_score', 'avg_t

Data types:
pa_score               int64
pa_time              float64
vp_score             float64
vp_time              float64
ls_score               int64
ls_time              float64
total_time             int64
pa_efficiency        float64
vp_efficiency        float64
ls_efficiency        float64
avg_score            float64
avg_time             float64
score_range          float64
time_range           float64
overall_efficiency   float64
time_penalty_index   float64
risk_level_encoded     int64
dtype: object

Risk level distribution:
risk_level_encoded
0    22
1    17
2    10
Name: count, dtype: int64
```

Split features and Targets

```
X = df.drop(columns=["risk_level_encoded"])
y = df["risk_level_encoded"]

print("Feature shape:", X.shape)
```

```
print("Target shape:", y.shape)
print("\nFeature columns:")
print(X.columns.tolist())
```

```
Feature shape: (49, 16)
Target shape: (49,)

Feature columns:
['pa_score', 'pa_time', 'vp_score', 'vp_time', 'ls_score', 'ls_time', 'total_time', 'pa_efficiency', 'vp_efficiency', 'ls_efficiency', 'avg_score', 'avg_t
```

Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("X_train:", X_train.shape)
print("X_test :", X_test.shape)
print("y_train:", y_train.shape)
print("y_test :", y_test.shape)
```

```
X_train: (39, 16)
X_test : (10, 16)
y_train: (39,)
y_test : (10,)
```

Create and train the Random Forest Model

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    random_state=42
)

model.fit(X_train, y_train)

print("Model trained successfully.")
```

```
Model trained successfully.
```

Predict on the test set

```
y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)

print("Predictions:")
print(y_pred)
```

```
Predictions:
[0 0 2 2 0 1 0 0 1 1]
```

Evaluate the Model

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("Accuracy:", accuracy)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
print("\nConfusion Matrix:")
print(cm)
```

```
Accuracy: 1.0

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         5
           1       1.00      1.00      1.00         3
           2       1.00      1.00      1.00         2

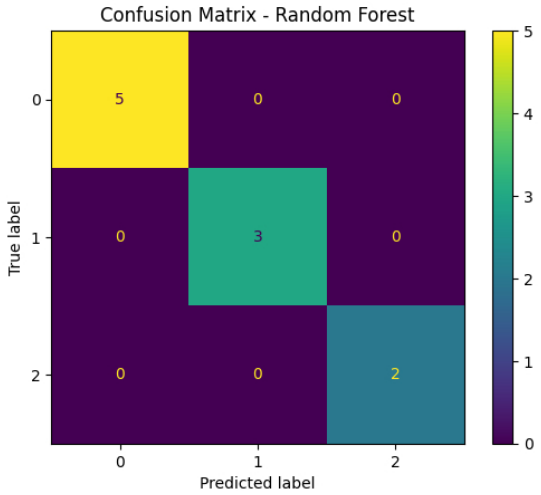
    accuracy                           1.00        10
   macro avg       1.00      1.00      1.00        10
weighted avg       1.00      1.00      1.00        10


Confusion Matrix:
[[5 0 0]
 [0 3 0]
 [0 0 2]]
```

Plot the confusion matrix

```
[ ]
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.title("Confusion Matrix - Random Forest")
plt.show()
```



Calculate the feature importance

```
[ ]
import pandas as pd

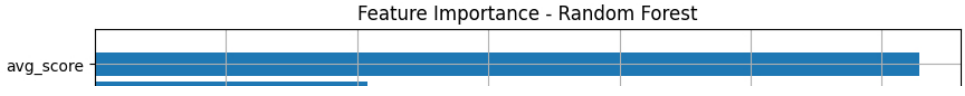
feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
}).sort_values(by="Importance", ascending=False)

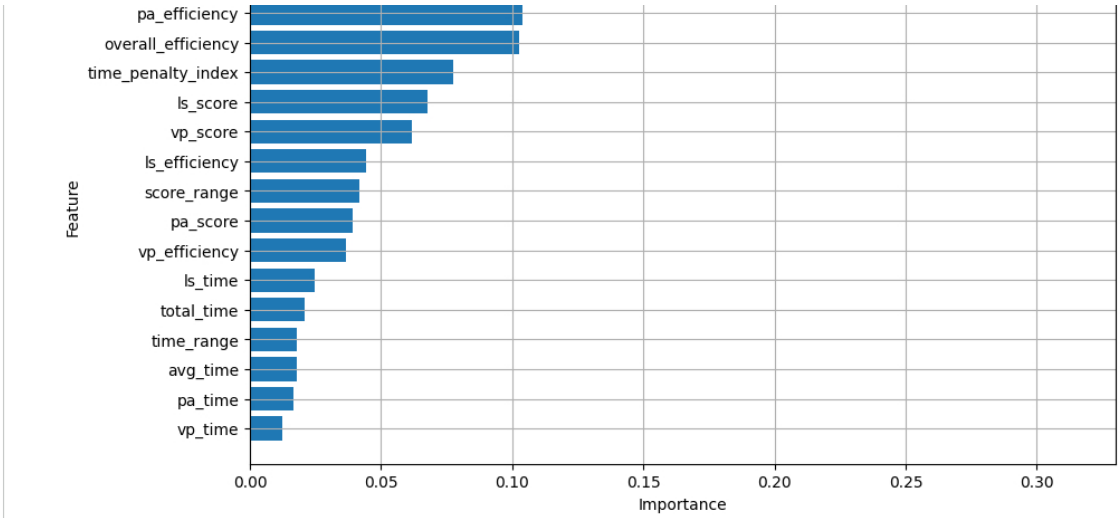
feature_importance
```

	Feature	Importance
10	avg_score	0.314347
7	pa_efficiency	0.103902
14	overall_efficiency	0.102834
15	time_penalty_index	0.077492
4	ls_score	0.067705
2	vp_score	0.061551
9	ls_efficiency	0.044130
12	score_range	0.041746
0	pa_score	0.039347
8	vp_efficiency	0.036499
5	ls_time	0.024730
6	total_time	0.020960
13	time_range	0.017877
11	avg_time	0.017813
1	pa_time	0.016765
3	vp_time	0.012302

Plot feature importance

```
[ ]
plt.figure(figsize=(10, 6))
plt.barh(feature_importance["Feature"], feature_importance["Importance"])
plt.gca().invert_yaxis()
plt.title("Feature Importance - Random Forest")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.grid(True)
plt.show()
```





Cross Validation

```
[ ]
from sklearn.model_selection import cross_val_score

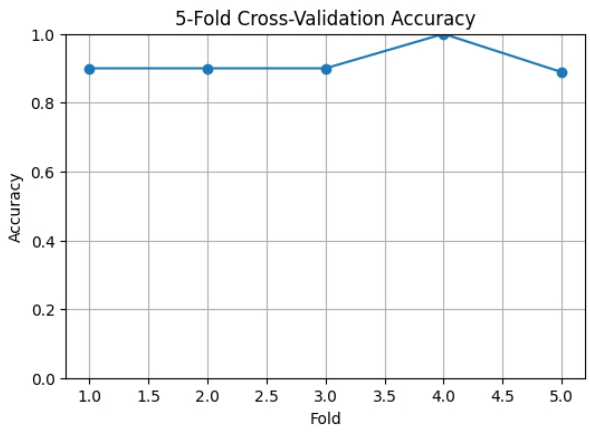
cv_scores = cross_val_score(
    model, X, y,
    cv=5,
    scoring="accuracy"
)

print("Cross-validation scores:", cv_scores)
print("Mean CV accuracy:", cv_scores.mean())
print("Std CV accuracy:", cv_scores.std())
```

```
Cross-validation scores: [0.9        0.9        0.9        1.         0.88888889]
Mean CV accuracy: 0.9177777777777777
Std CV accuracy: 0.0413357227505295
```

Plot cross validation Scores

```
[ ]
plt.figure(figsize=(6, 4))
plt.plot(range(1, len(cv_scores) + 1), cv_scores, marker="o")
plt.title("5-Fold Cross-Validation Accuracy")
plt.xlabel("Fold")
plt.ylabel("Accuracy")
plt.ylim(0, 1)
plt.grid(True)
plt.show()
```



Build actual Vs. Predected table

```
[ ]
results_df = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred
})

results_df
```

	Actual	Predicted
0	0	0
1	0	0

2	2	2
3	2	2
4	0	0
5	1	1
6	0	0
7	0	0
8	1	1
9	1	1

Convereted numbers into readable labels

```
label_map = {
    0: "Low",
    1: "Moderate",
    2: "High"
}

results_df["Actual_Label"] = results_df["Actual"].map(label_map)
results_df["Predicted_Label"] = results_df["Predicted"].map(label_map)

results_df
```

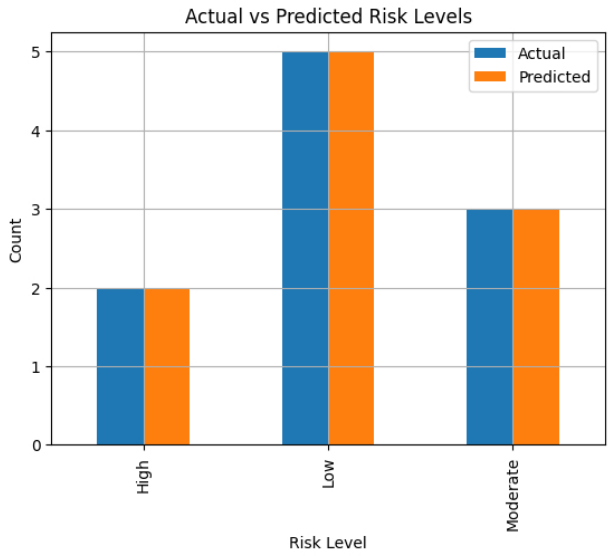
	Actual	Predicted	Actual_Label	Predicted_Label
0	0	0	Low	Low
1	0	0	Low	Low
2	2	2	High	High
3	2	2	High	High
4	0	0	Low	Low
5	1	1	Moderate	Moderate
6	0	0	Low	Low
7	0	0	Low	Low
8	1	1	Moderate	Moderate
9	1	1	Moderate	Moderate

Compared actual and predicted counts

```
actual_counts = results_df["Actual_Label"].value_counts().sort_index()
pred_counts = results_df["Predicted_Label"].value_counts().sort_index()

comparison_df = pd.DataFrame({
    "Actual": actual_counts,
    "Predicted": pred_counts
}).fillna(0)

comparison_df.plot(kind="bar")
plt.title("Actual vs Predicted Risk Levels")
plt.xlabel("Risk Level")
plt.ylabel("Count")
plt.grid(True)
plt.show()
```



Save the trained model

```
import joblib

joblib.dump(model, "random_forest_risk_model_1.pkl")
print("Model saved as random_forest_risk_model_1.pkl")
```

```
Model saved as random_forest_risk_model_1.pkl
```

Download the model file

```
from google.colab import files
files.download("random_forest_risk_model_1.pkl")
```

Predict for the full data set

```
full_predictions = model.predict(X)

full_results = df.copy()
full_results["predicted_risk_level_encoded"] = full_predictions
full_results["predicted_risk_level"] = full_results["predicted_risk_level_encoded"].map(label_map)

full_results.head()
```

	pa_score	pa_time	vp_score	vp_time	ls_score	ls_time	total_time	pa_efficiency	vp_efficiency	ls_efficiency	avg_score	avg_time	score_range	tim
0	20	11.13	50.000000	11.21	50	17.24	192	1.796945	4.460303	2.900232	40.000000	13.193333	30.000000	
1	0	24.28	16.666667	13.52	75	15.00	263	0.000000	1.232742	5.000000	30.555556	17.600000	75.000000	
2	20	10.65	33.333333	13.30	50	10.59	175	1.877934	2.506265	4.721435	34.444444	11.513333	30.000000	
3	60	23.03	33.333333	22.14	50	12.68	299	2.605297	1.505571	3.943217	47.777778	19.283333	26.666667	
4	40	22.15	50.000000	31.34	25	18.65	373	1.805869	1.595405	1.340483	38.333333	24.046667	25.000000	

Saved the predictions report

```
full_results.to_csv("rf_predictions_report.csv", index=False)
print("Saved rf_predictions_report.csv")
```

```
Saved rf_predictions_report.csv
```

Download predictions report

```
from google.colab import files
files.download("rf_predictions_report.csv")
```

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Test the model with sample students

```
import pandas as pd

# Simple sample student data
sample = pd.DataFrame([{
    "pa_score": 80,
    "pa_time": 10.0,
    "vp_score": 90,
    "vp_time": 10.0,
    "ls_score": 95,
    "ls_time": 10.0,
    "total_time": 30
}])

# Create required extra features
sample["pa_efficiency"] = sample["pa_score"] / sample["pa_time"]
sample["vp_efficiency"] = sample["vp_score"] / sample["vp_time"]
sample["ls_efficiency"] = sample["ls_score"] / sample["ls_time"]

sample["avg_score"] = sample[["pa_score", "vp_score", "ls_score"]].mean(axis=1)
sample["avg_time"] = sample[["pa_time", "vp_time", "ls_time"]].mean(axis=1)
sample["score_range"] = sample[["pa_score", "vp_score", "ls_score"]].max(axis=1) - sample[["pa_score", "vp_score", "ls_score"]].min(axis=1)
sample["time_range"] = sample[["pa_time", "vp_time", "ls_time"]].max(axis=1) - sample[["pa_time", "vp_time", "ls_time"]].min(axis=1)
```

```
sample["overall_efficiency"] = sample["avg_score"] / sample["avg_time"]
sample["time_penalty_index"] = sample["total_time"] / sample["avg_score"]

# Predict
prediction = model.predict(sample)[0]

# Convert number to label
label_map = {0: "Low", 1: "Moderate", 2: "High"}

print("Predicted Risk Level:", label_map[prediction])
```

```
Predicted Risk Level: Low
```

Colab paid products - Cancel contracts here